In [6]:
```
# ============================================================
# EXERCISE 2: DATA DISCOVERY AND PREPARATION
# CUSTOMER CHURN DATASET
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder


# ============================================================
# 1. LOAD DATASET
# ============================================================

df = pd.read_csv(
    r"C:\Users\DELL\Downloads\archive\customer_churn_dataset-training-master.csv"
)

print("=" * 60)
print("DATASET LOADED SUCCESSFULLY")
print("=" * 60)

print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nStatistical Summary:")
print(df.describe(include="all"))

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())


# ============================================================
# 2. DATA CLEANING
# ============================================================

# Remove duplicate rows
df = df.drop_duplicates()

# Check missing values
print("\nShape After Removing Duplicates:")
print(df.shape)

# Remove CustomerID because it is only an identifier
if "CustomerID" in df.columns:
    df = df.drop("CustomerID", axis=1)
```

```
print("\nColumns After Removing CustomerID:")
print(df.columns.tolist())

# Check missing values again
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())


# ============================================================
# 3. DISTRIBUTION OF CUSTOMER CHURN
# ============================================================

print("\n" + "=" * 60)
print("CUSTOMER CHURN DISTRIBUTION")
print("=" * 60)

print(df["Churn"].value_counts())

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Churn")
plt.title("Distribution of Customer Churn")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
# 4. DISTRIBUTION OF GENDER
# ============================================================

print("\n" + "=" * 60)
print("GENDER DISTRIBUTION")
print("=" * 60)

print(df["Gender"].value_counts())

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Gender")
plt.title("Distribution of Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
# 5. DISTRIBUTION OF CONTRACT TYPE
# ============================================================

print("\n" + "=" * 60)
print("CONTRACT LENGTH DISTRIBUTION")
print("=" * 60)

print(df["Contract Length"].value_counts())

plt.figure(figsize=(7, 4))
sns.countplot(data=df, x="Contract Length")
plt.title("Distribution of Contract Length")
plt.xlabel("Contract Length")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
```

```
# 6. DISTRIBUTION OF PAYMENT / SUBSCRIPTION TYPE
# ============================================================

print("\n" + "=" * 60)
print("SUBSCRIPTION TYPE DISTRIBUTION")
print("=" * 60)

print(df["Subscription Type"].value_counts())

plt.figure(figsize=(7, 4))
sns.countplot(data=df, x="Subscription Type")
plt.title("Distribution of Subscription Type")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
# 7. HISTOGRAMS AND BOXPLOTS
# ============================================================

numerical_columns = [
    "Age",
    "Tenure",
    "Usage Frequency",
    "Support Calls",
    "Payment Delay",
    "Total Spend",
    "Last Interaction"
]

for column in numerical_columns:

    # Histogram
    plt.figure(figsize=(7, 4))
    plt.hist(df[column], bins=30)
    plt.title("Histogram of " + column)
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.show()

    # Boxplot
    plt.figure(figsize=(7, 3))
    sns.boxplot(x=df[column])
    plt.title("Boxplot of " + column)
    plt.xlabel(column)
    plt.show()


# ============================================================
# 8. OUTLIER DETECTION USING IQR
# ============================================================

print("\n" + "=" * 60)
print("OUTLIER DETECTION USING IQR")
print("=" * 60)

for column in numerical_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]

    print("\nFeature:", column)
    print("Q1:", Q1)
    print("Q3:", Q3)
    print("IQR:", IQR)
    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)
    print("Number of Outliers:", len(outliers))


# ============================================================
# 9. CONTRACT LENGTH VS CHURN
# ============================================================

print("\n" + "=" * 60)
print("CONTRACT LENGTH VS CHURN")
print("=" * 60)

contract_churn = pd.crosstab(
    df["Contract Length"],
    df["Churn"],
    normalize="index"
) * 100

print(contract_churn)

plt.figure(figsize=(8, 5))

sns.countplot(
    data=df,
    x="Contract Length",
    hue="Churn"
)

plt.title("Contract Length vs Customer Churn")
plt.xlabel("Contract Length")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
# 10. SUBSCRIPTION TYPE VS CHURN
# ============================================================

print("\n" + "=" * 60)
print("SUBSCRIPTION TYPE VS CHURN")
print("=" * 60)

subscription_churn = pd.crosstab(
    df["Subscription Type"],
    df["Churn"],
    normalize="index"
) * 100

print(subscription_churn)

plt.figure(figsize=(8, 5))
```

```
sns.countplot(
    data=df,
    x="Subscription Type",
    hue="Churn"
)

plt.title("Subscription Type vs Customer Churn")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")
plt.show()


# ============================================================
# 11. TENURE VS CHURN
# ============================================================

plt.figure(figsize=(7, 5))

sns.boxplot(
    data=df,
    x="Churn",
    y="Tenure"
)

plt.title("Tenure vs Customer Churn")
plt.xlabel("Churn")
plt.ylabel("Tenure")

plt.show()


# ============================================================
# 12. TOTAL SPEND VS CHURN
# ============================================================

plt.figure(figsize=(7, 5))

sns.boxplot(
    data=df,
    x="Churn",
    y="Total Spend"
)

plt.title("Total Spend vs Customer Churn")
plt.xlabel("Churn")
plt.ylabel("Total Spend")

plt.show()


# ============================================================
# 13. SUPPORT CALLS VS CHURN
# ============================================================

plt.figure(figsize=(7, 5))

sns.boxplot(
    data=df,
    x="Churn",
    y="Support Calls"
)

plt.title("Support Calls vs Customer Churn")
```

```
plt.xlabel("Churn")
plt.ylabel("Support Calls")

plt.show()


# ============================================================
# 14. PAYMENT DELAY VS CHURN
# ============================================================

plt.figure(figsize=(7, 5))

sns.boxplot(
    data=df,
    x="Churn",
    y="Payment Delay"
)

plt.title("Payment Delay vs Customer Churn")
plt.xlabel("Churn")
plt.ylabel("Payment Delay")

plt.show()


# ============================================================
# 15. SCATTER PLOT: TENURE VS TOTAL SPEND
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Tenure",
    y="Total Spend",
    hue="Churn"
)

plt.title("Tenure vs Total Spend by Churn")
plt.xlabel("Tenure")
plt.ylabel("Total Spend")

plt.show()


# ============================================================
# TASK 6: ANALYTICAL QUESTIONS
# ============================================================

print("\n" + "=" * 60)
print("ANALYTICAL QUESTIONS")
print("=" * 60)


# ------------------------------------------------------------
# 1. Which contract type has the highest customer churn?
# ------------------------------------------------------------

print("\n1. CONTRACT TYPE WITH HIGHEST CHURN")

contract_churn = pd.crosstab(
    df["Contract Length"],
    df["Churn"],
    normalize="index"
```

```
) * 100

print(contract_churn)

if 1 in contract_churn.columns:
    highest_contract = contract_churn[1].idxmax()

    print(
        "Contract Length with highest churn:",
        highest_contract
    )

    print(
        "Churn Rate:",
        round(contract_churn[1].max(), 2),
        "%"
    )

elif "Yes" in contract_churn.columns:
    highest_contract = contract_churn["Yes"].idxmax()

    print(
        "Contract Length with highest churn:",
        highest_contract
    )


# ------------------------------------------------------------
# 2. Which subscription type has highest churn?
# ------------------------------------------------------------

print("\n2. SUBSCRIPTION TYPE WITH HIGHEST CHURN")

subscription_churn = pd.crosstab(
    df["Subscription Type"],
    df["Churn"],
    normalize="index"
) * 100

print(subscription_churn)

if 1 in subscription_churn.columns:

    highest_subscription = subscription_churn[1].idxmax()

    print(
        "Subscription Type with highest churn:",
        highest_subscription
    )


# ------------------------------------------------------------
# 3. Do customers with shorter tenure churn more often?
# ------------------------------------------------------------

print("\n3. TENURE AND CHURN")

print(
    df.groupby("Churn")["Tenure"].mean()
)

print("\nMedian Tenure:")
print(
    df.groupby("Churn")["Tenure"].median()
```

```
)


# ------------------------------------------------------------
# 4. Which customer group has highest total spend?
# ------------------------------------------------------------

print("\n4. AVERAGE TOTAL SPEND")

print(
    df.groupby("Subscription Type")["Total Spend"]
    .mean()
    .sort_values(ascending=False)
)

print("\nAverage Total Spend by Contract Length:")

print(
    df.groupby("Contract Length")["Total Spend"]
    .mean()
    .sort_values(ascending=False)
)


# ------------------------------------------------------------
# 5. Which group has highest churn?
# ------------------------------------------------------------

print("\n5. CHURN BY SUBSCRIPTION TYPE")

print(
    pd.crosstab(
        df["Subscription Type"],
        df["Churn"],
        normalize="index"
    ) * 100
)


# ------------------------------------------------------------
# 6. Is there a relationship between spending and churn?
# ------------------------------------------------------------

print("\n6. TOTAL SPEND AND CHURN")

print(
    df.groupby("Churn")["Total Spend"].mean()
)

print("\nMedian Total Spend by Churn:")

print(
    df.groupby("Churn")["Total Spend"].median()
)


# ------------------------------------------------------------
# 7. Strongest indicators of churn
# ------------------------------------------------------------

print("\n7. POTENTIAL STRONG INDICATORS OF CHURN")

print("\nAverage Support Calls by Churn:")
print(
```

```
    df.groupby("Churn")["Support Calls"].mean()
)

print("\nAverage Payment Delay by Churn:")
print(
    df.groupby("Churn")["Payment Delay"].mean()
)

print("\nAverage Tenure by Churn:")
print(
    df.groupby("Churn")["Tenure"].mean()
)

print("\nAverage Usage Frequency by Churn:")
print(
    df.groupby("Churn")["Usage Frequency"].mean()
)

print("\nAverage Total Spend by Churn:")
print(
    df.groupby("Churn")["Total Spend"].mean()
)


# ============================================================
# 16. CORRELATION ANALYSIS
# ============================================================

print("\n" + "=" * 60)
print("CORRELATION WITH CHURN")
print("=" * 60)

correlation_data = df[
    [
        "Age",
        "Tenure",
        "Usage Frequency",
        "Support Calls",
        "Payment Delay",
        "Total Spend",
        "Last Interaction",
        "Churn"
    ]
]

correlation = correlation_data.corr()

print(correlation["Churn"].sort_values(ascending=False))


# Correlation heatmap

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()
```

```
# ============================================================
# TASK 7: TRAIN TEST SPLIT
# ============================================================

print("\n" + "=" * 60)
print("TRAIN TEST SPLIT")
print("=" * 60)

# Check missing values in Churn
print("\nMissing values in Churn:")
print(df["Churn"].isnull().sum())

# Remove rows where Churn is missing
df = df.dropna(subset=["Churn"])

# Separate features and target
X = df.drop("Churn", axis=1)
y = df["Churn"]

# Check target values
print("\nChurn values:")
print(y.value_counts())

print("\nMissing values in target after cleaning:")
print(y.isnull().sum())

# Convert categorical features into numerical variables
X = pd.get_dummies(
    X,
    drop_first=True
)

# Split dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Features Shape:")
print(X_train.shape)

print("\nTesting Features Shape:")
print(X_test.shape)

print("\nTraining Target Shape:")
print(y_train.shape)

print("\nTesting Target Shape:")
print(y_test.shape)


# ============================================================
# TASK 8: SAVE PREPROCESSED DATASET
# ============================================================

processed_df = X.copy()

processed_df["Churn"] = y

output_path = r"C:\Users\DELL\Downloads\CustomerChurn_Preprocessed.csv"
```

```
processed_df.to_csv(
    output_path,
    index=False
)

print("\n" + "=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)

print("\nFinal Processed Dataset Shape:")
print(processed_df.shape)

print("\nTraining Dataset Shape:")
print(X_train.shape)

print("\nTesting Dataset Shape:")
print(X_test.shape)

print("\nFile Saved Successfully At:")
print(output_path)
```

```
============================================================
DATASET LOADED SUCCESSFULLY
============================================================

First 5 Rows:
   CustomerID   Age  Gender  Tenure  Usage Frequency  Support Calls  \
0         2.0  30.0  Female    39.0             14.0            5.0
1         3.0  65.0  Female    49.0              1.0           10.0
2         4.0  55.0  Female    14.0              4.0            6.0
3         5.0  58.0    Male    38.0             21.0            7.0
4         6.0  23.0    Male    32.0             20.0            5.0

   Payment Delay Subscription Type Contract Length  Total Spend  \
0           18.0          Standard          Annual        932.0
1            8.0             Basic         Monthly        557.0
2           18.0             Basic       Quarterly        185.0
3            7.0          Standard         Monthly        396.0
4            8.0             Basic         Monthly        617.0

   Last Interaction  Churn
0              17.0    1.0
1               6.0    1.0
2               3.0    1.0
3              29.0    1.0
4              20.0    1.0

Dataset Shape:
(440833, 12)

Column Names:
['CustomerID', 'Age', 'Gender', 'Tenure', 'Usage Frequency', 'Support Calls', 'Pay
ment Delay', 'Subscription Type', 'Contract Length', 'Total Spend', 'Last Interact
ion', 'Churn']

Data Types:
CustomerID           float64
Age                  float64
Gender                object
Tenure               float64
Usage Frequency      float64
Support Calls        float64
Payment Delay        float64
Subscription Type     object
Contract Length       object
Total Spend          float64
Last Interaction     float64
Churn                float64
dtype: object

Statistical Summary:
          CustomerID            Age  Gender         Tenure  Usage Frequency  \
count   440832.000000  440832.000000  440832  440832.000000    440832.000000
unique            NaN            NaN       2            NaN              NaN
top               NaN            NaN    Male            NaN              NaN
freq              NaN            NaN  250252            NaN              NaN
mean    225398.667955      39.373153     NaN      31.256336        15.807494
std     129531.918550      12.442369     NaN      17.255727         8.586242
min          2.000000      18.000000     NaN       1.000000         1.000000
25%     113621.750000      29.000000     NaN      16.000000         9.000000
50%     226125.500000      39.000000     NaN      32.000000        16.000000
75%     337739.250000      48.000000     NaN      46.000000        23.000000
max     449999.000000      65.000000     NaN      60.000000        30.000000

        Support Calls  Payment Delay Subscription Type Contract Length  \
```

```
count   440832.000000  440832.000000            440832          440832
unique            NaN            NaN                 3               3
top               NaN            NaN          Standard          Annual
freq              NaN            NaN            149128          177198
mean         3.604437      12.965722               NaN             NaN
std          3.070218       8.258063               NaN             NaN
min          0.000000       0.000000               NaN             NaN
25%          1.000000       6.000000               NaN             NaN
50%          3.000000      12.000000               NaN             NaN
75%          6.000000      19.000000               NaN             NaN
max         10.000000      30.000000               NaN             NaN

          Total Spend  Last Interaction          Churn
count   440832.000000     440832.000000  440832.000000
unique            NaN               NaN            NaN
top               NaN               NaN            NaN
freq              NaN               NaN            NaN
mean       631.616223         14.480868       0.567107
std        240.803001          8.596208       0.495477
min        100.000000          1.000000       0.000000
25%        480.000000          7.000000       0.000000
50%        661.000000         14.000000       1.000000
75%        830.000000         22.000000       1.000000
max       1000.000000         30.000000       1.000000

Missing Values:
CustomerID           1
Age                  1
Gender               1
Tenure               1
Usage Frequency      1
Support Calls        1
Payment Delay        1
Subscription Type    1
Contract Length      1
Total Spend          1
Last Interaction     1
Churn                1
dtype: int64

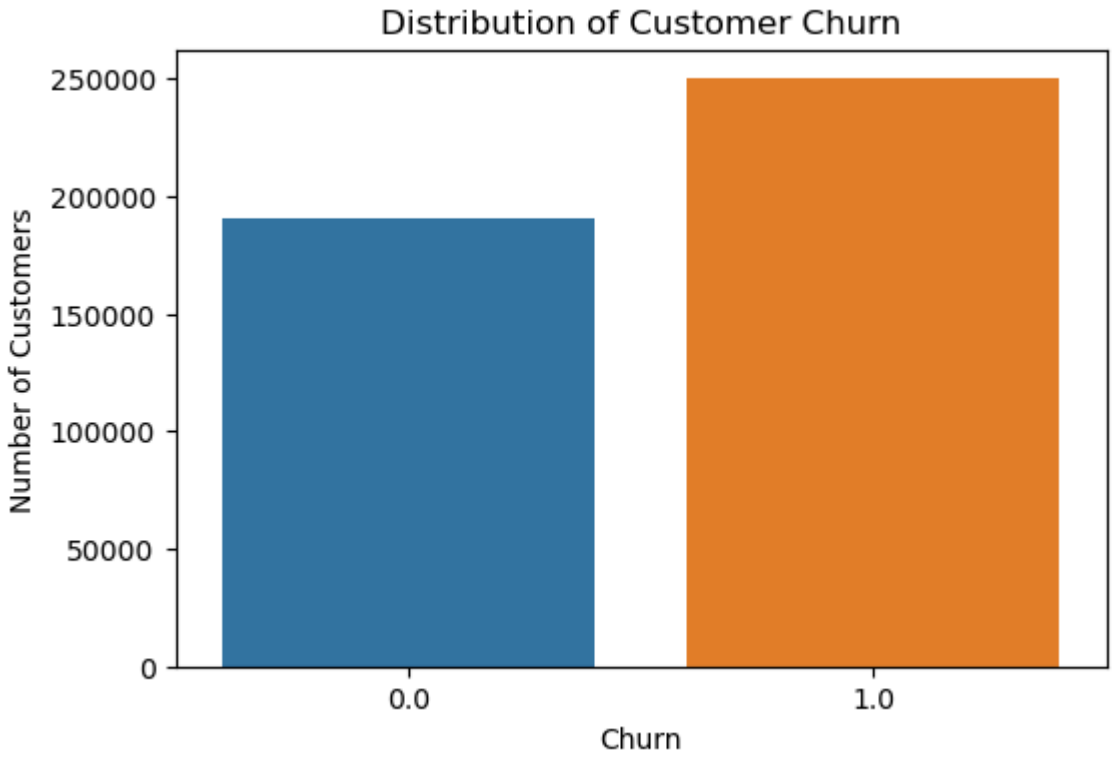
Duplicate Rows:
0

Shape After Removing Duplicates:
(440833, 12)

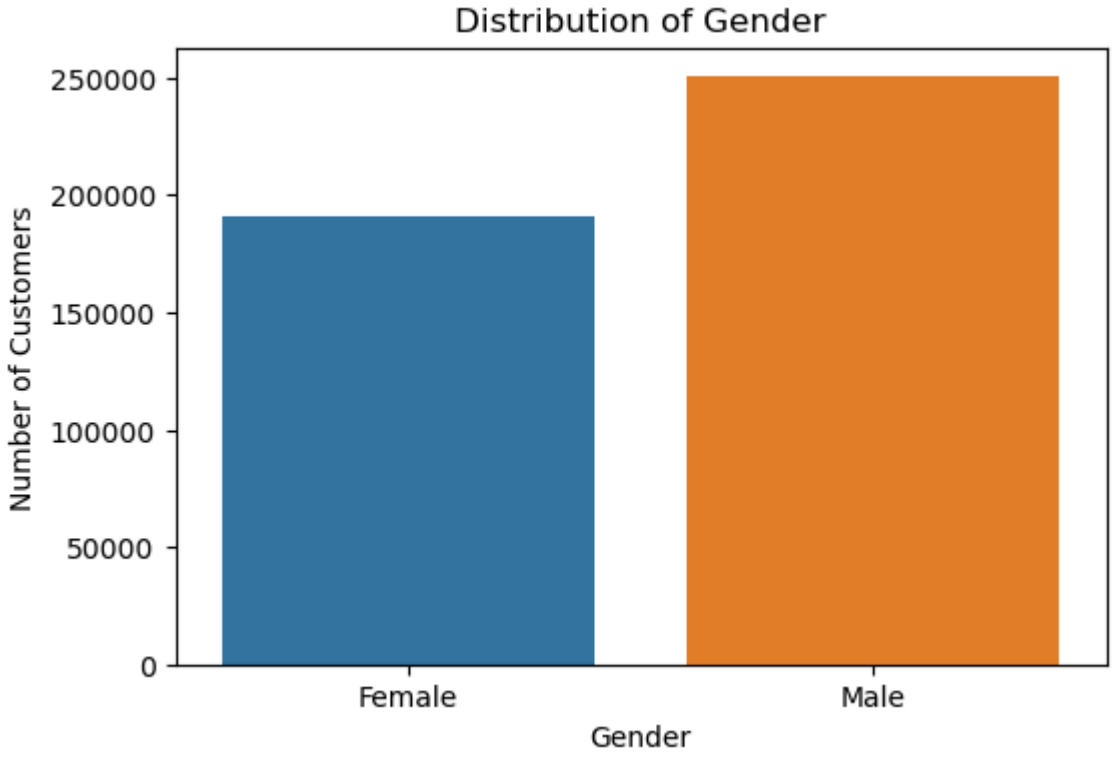
Columns After Removing CustomerID:
['Age', 'Gender', 'Tenure', 'Usage Frequency', 'Support Calls', 'Payment Delay',
'Subscription Type', 'Contract Length', 'Total Spend', 'Last Interaction', 'Chur
n']

Missing Values After Cleaning:
Age                  1
Gender               1
Tenure               1
Usage Frequency      1
Support Calls        1
Payment Delay        1
Subscription Type    1
Contract Length      1
Total Spend          1
Last Interaction     1
Churn                1
dtype: int64
```

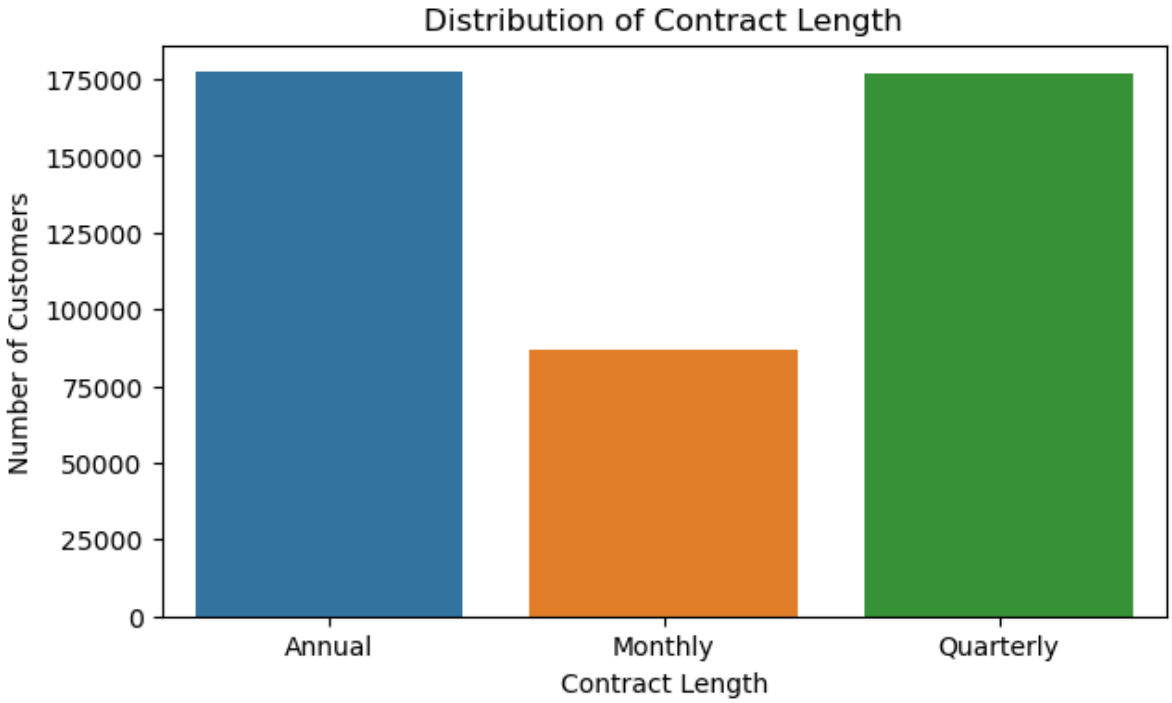
```
============================================================
CUSTOMER CHURN DISTRIBUTION
============================================================
1.0    249999
0.0    190833
Name: Churn, dtype: int64
```



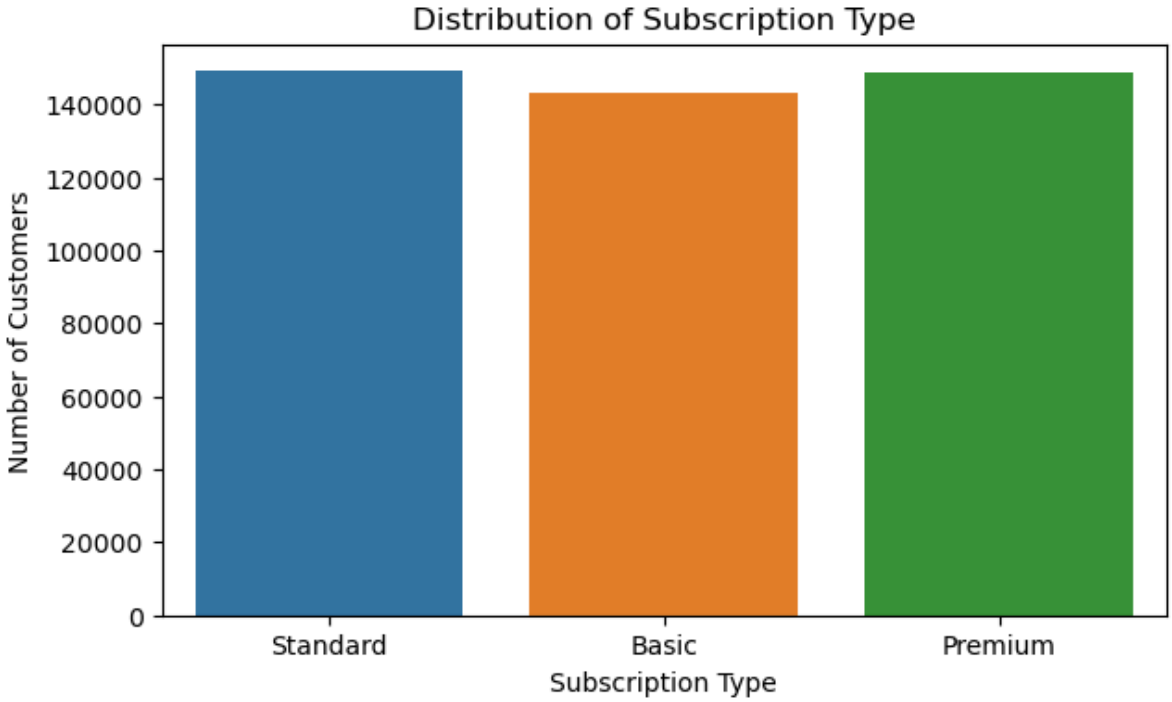

```
============================================================
GENDER DISTRIBUTION
============================================================
Male      250252
Female    190580
Name: Gender, dtype: int64
```

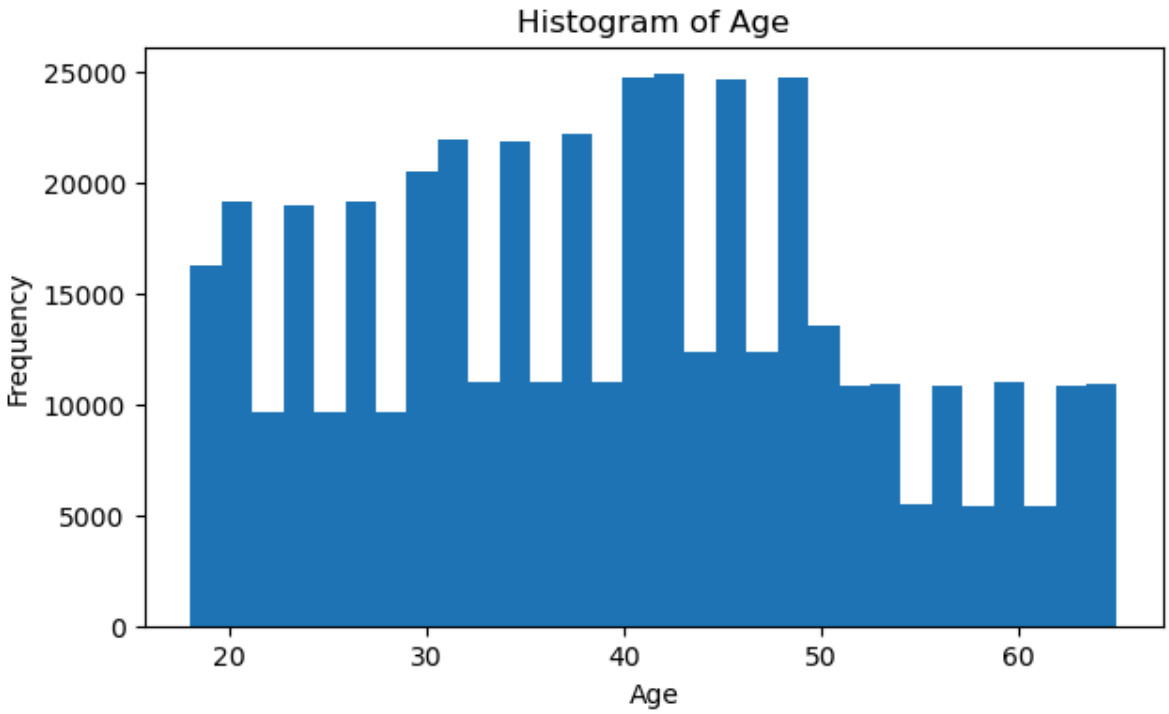


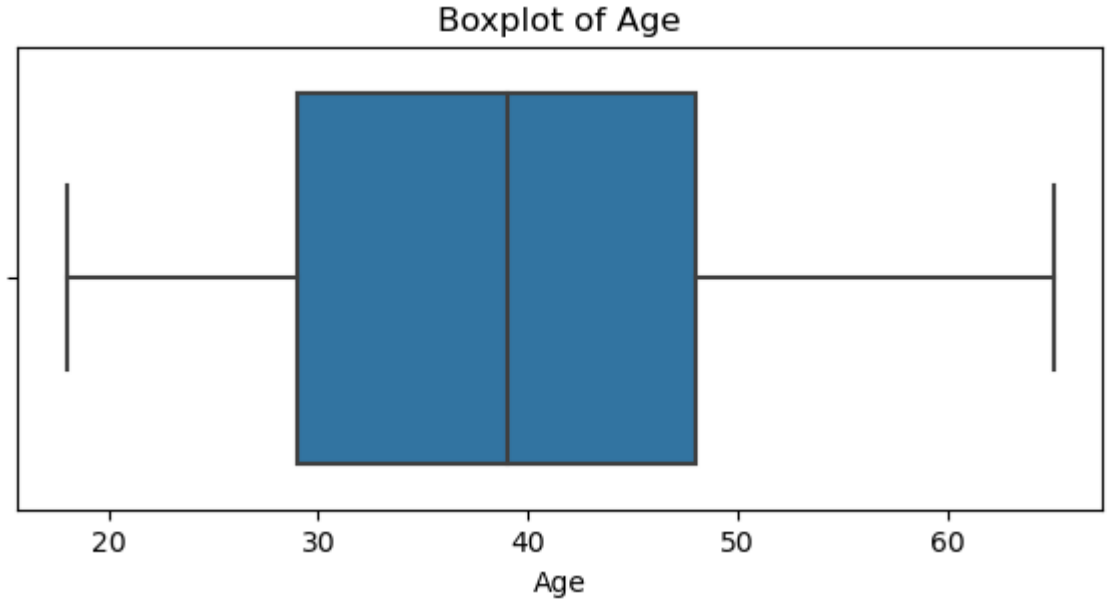
```
============================================================
CONTRACT LENGTH DISTRIBUTION
============================================================
Annual       177198
Quarterly    176530
Monthly       87104
Name: Contract Length, dtype: int64
```

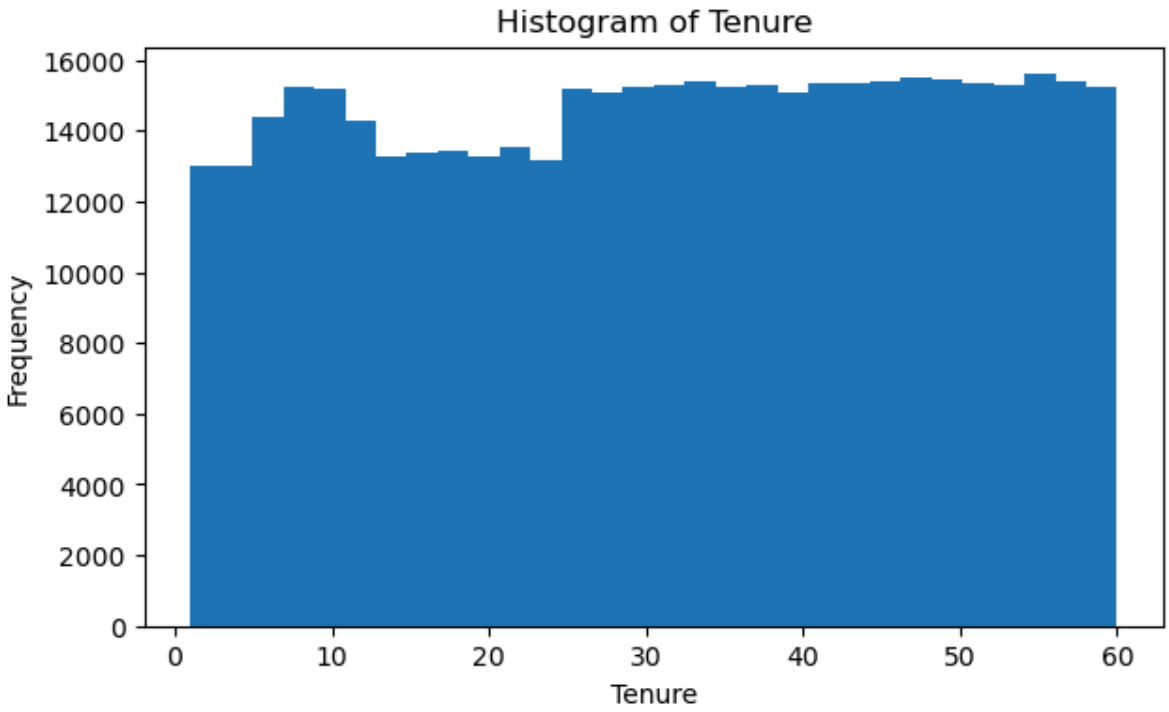


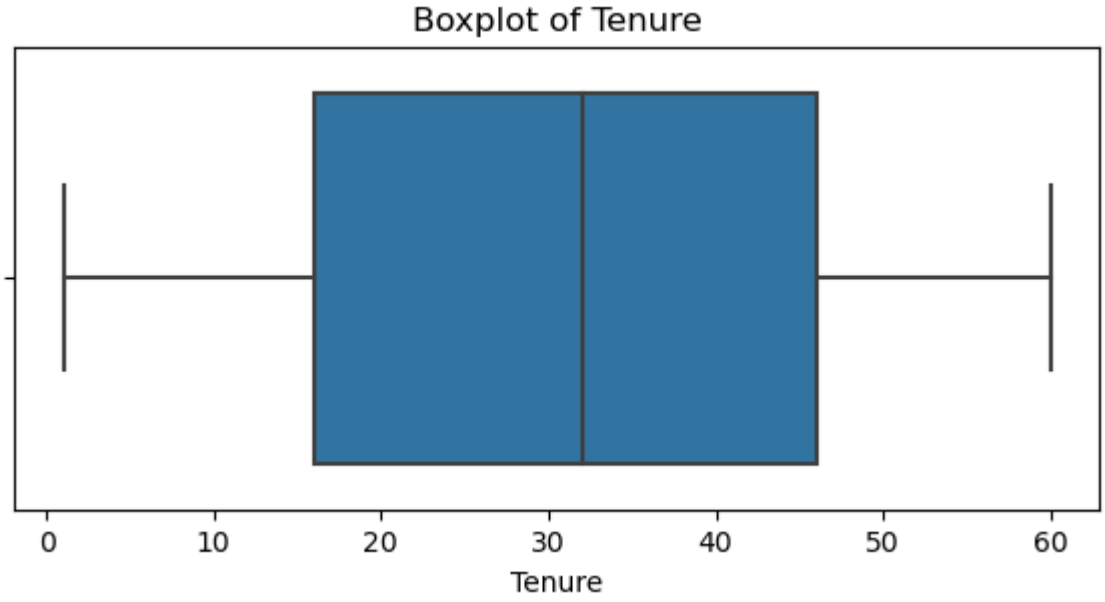

```
============================================================
SUBSCRIPTION TYPE DISTRIBUTION
============================================================
Standard    149128
Premium     148678
Basic       143026
Name: Subscription Type, dtype: int64
```

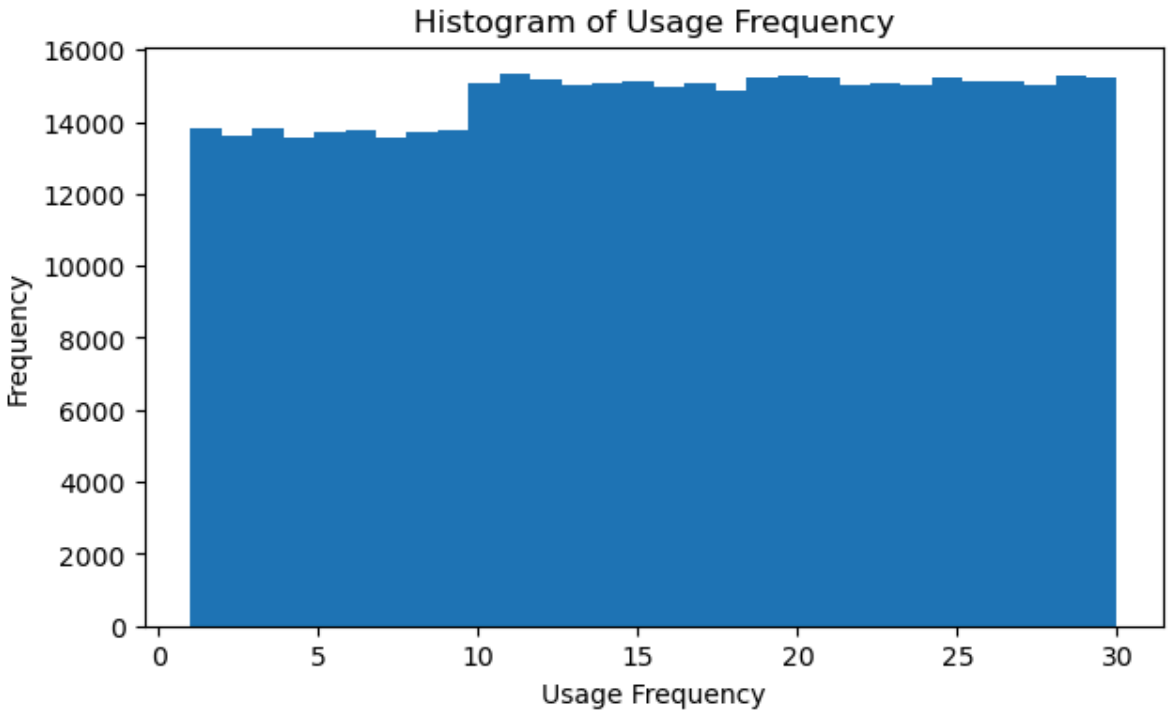


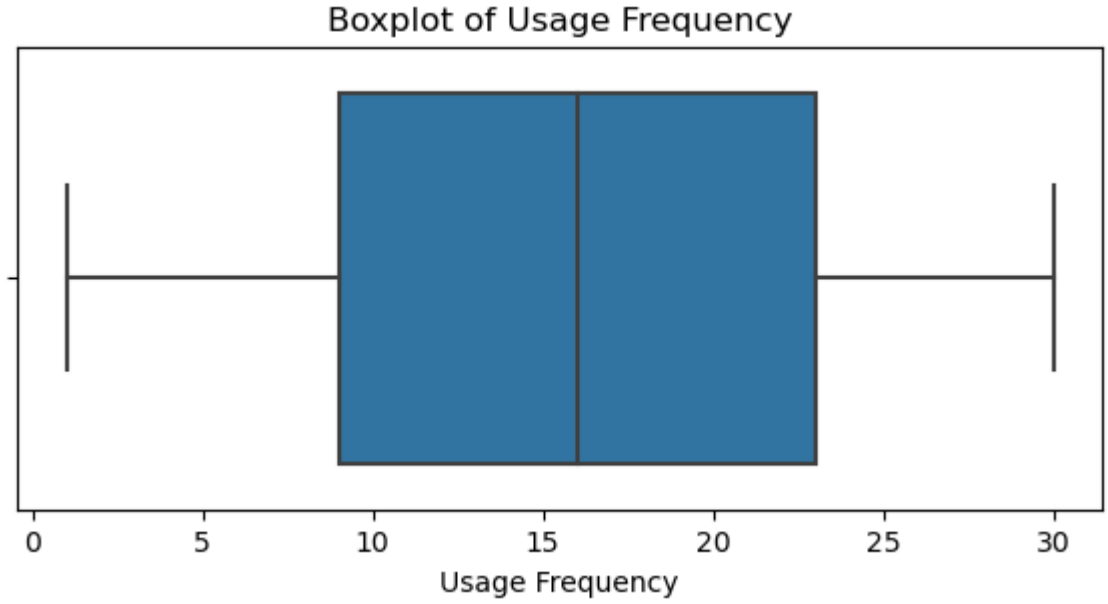

Histogram of Age
Frequency
25000
20000
15000
10000
5000
0
20
30
40
50
60
Age

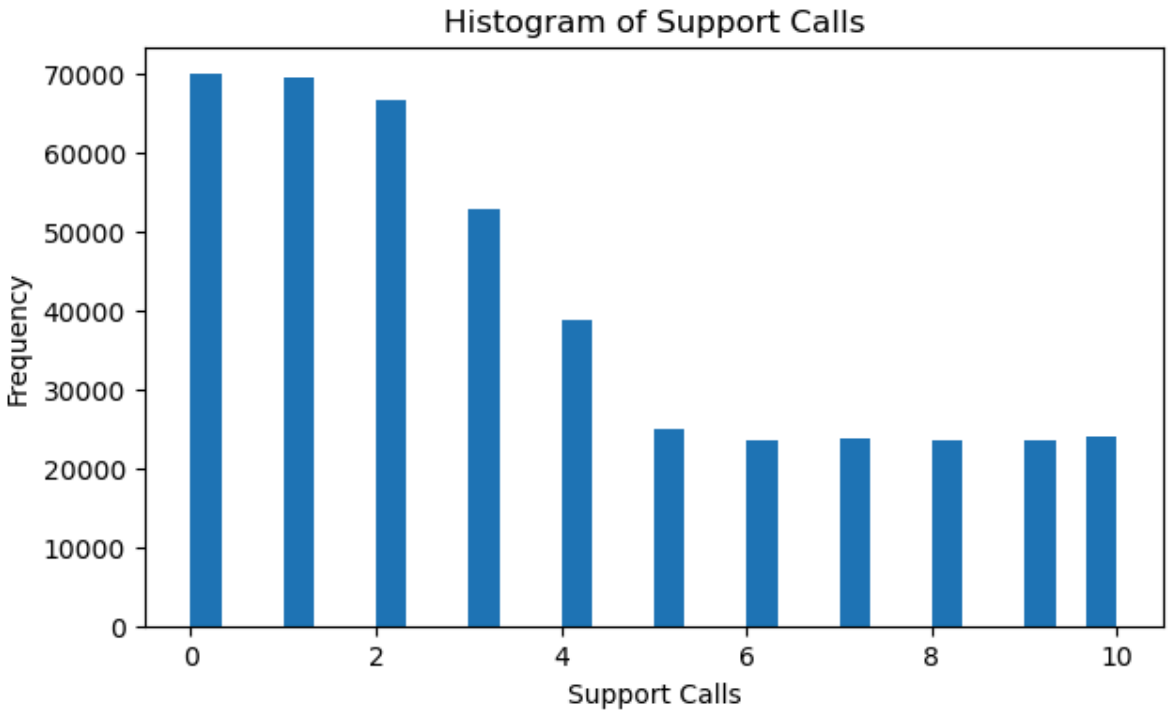

Boxplot of Age
20
30
40
50
60
Age

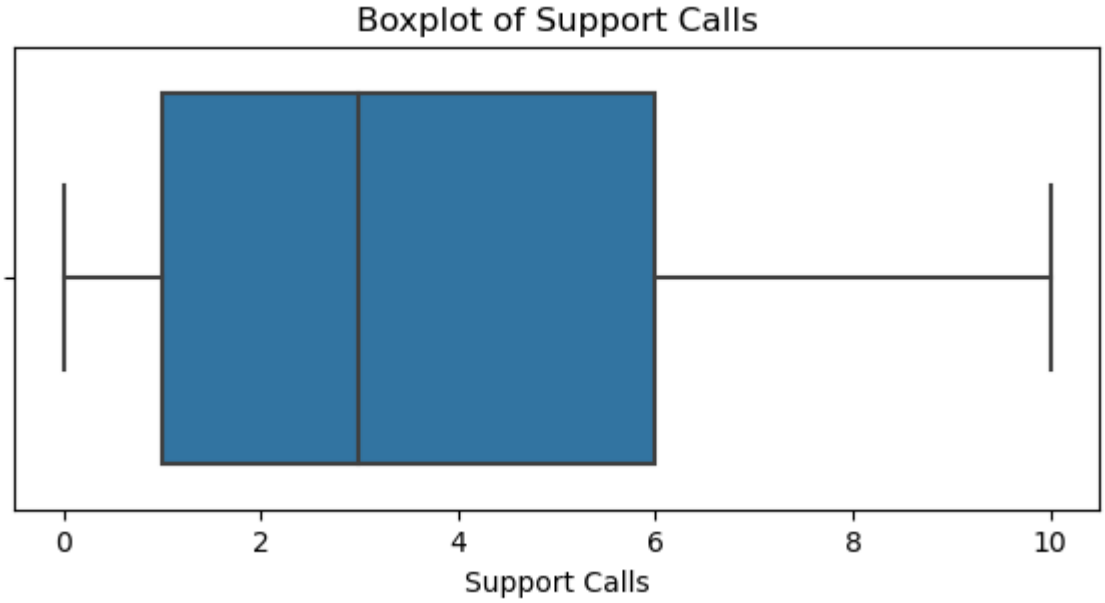

Histogram of Tenure
Frequency
16000
14000
12000
10000
8000
6000
4000
2000
0
0
10
20
30
40
50
60
Tenure

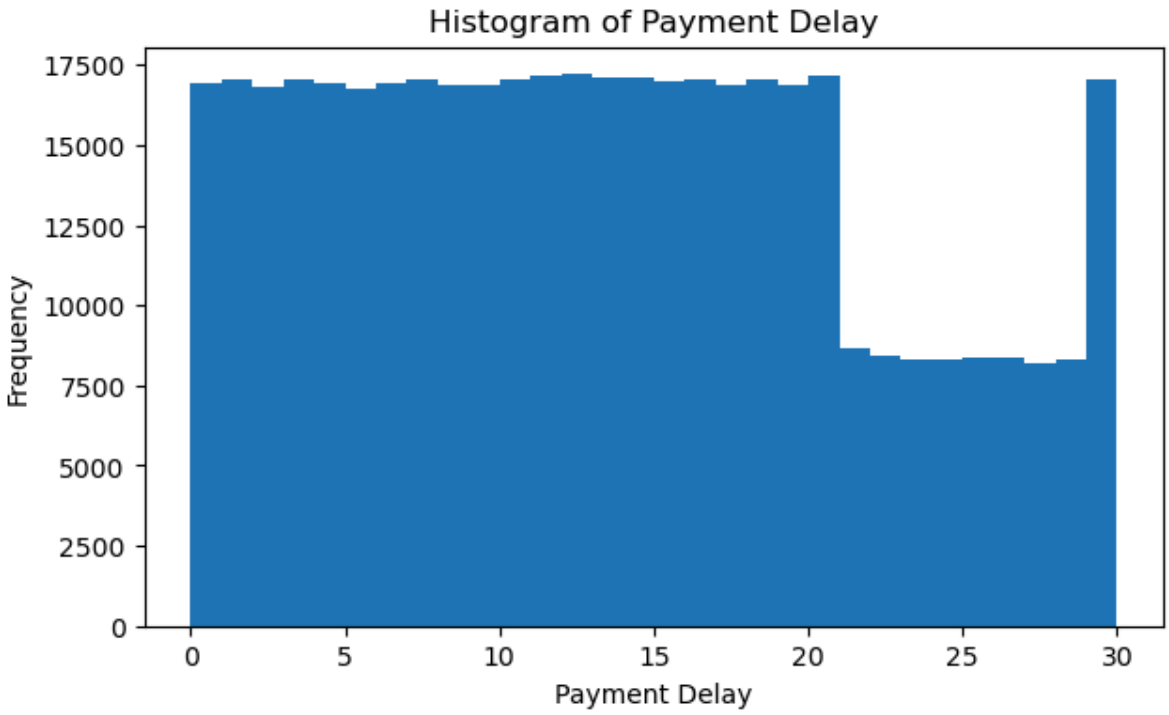

Boxplot of Tenure
0
10
20
30
40
50
60
Tenure

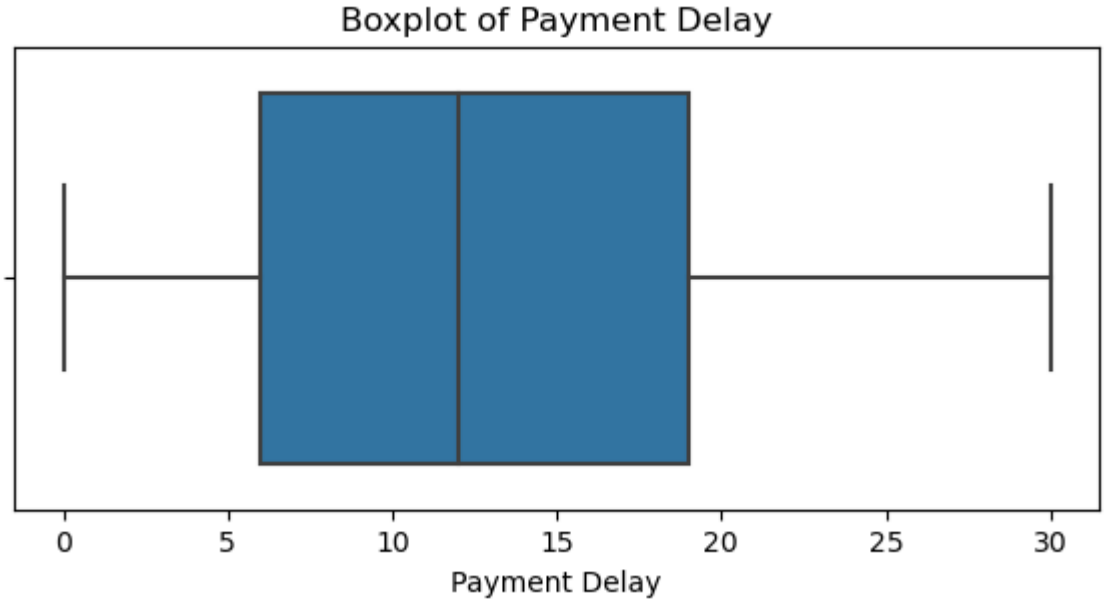

Histogram of Usage Frequency
Frequency
Usage Frequency
0
2000
4000
6000
8000
10000
12000
14000
16000
0
5
10
15
20
25
30

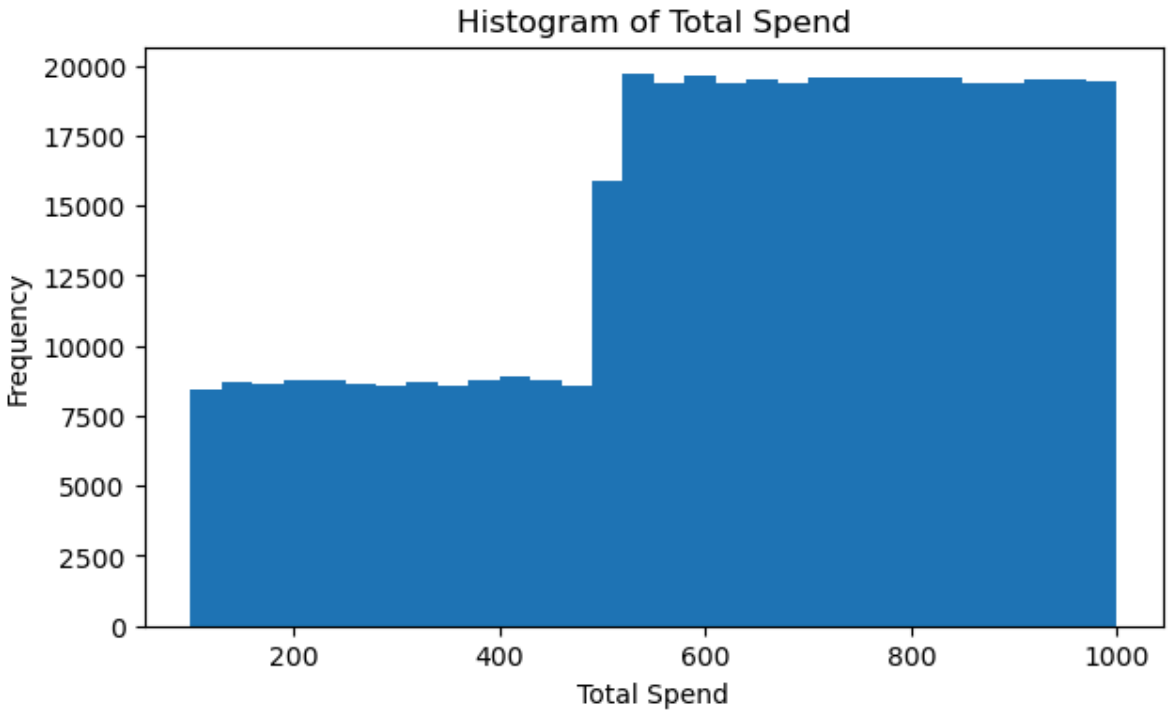

Boxplot of Usage Frequency
Usage Frequency
0
5
10
15
20
25
30

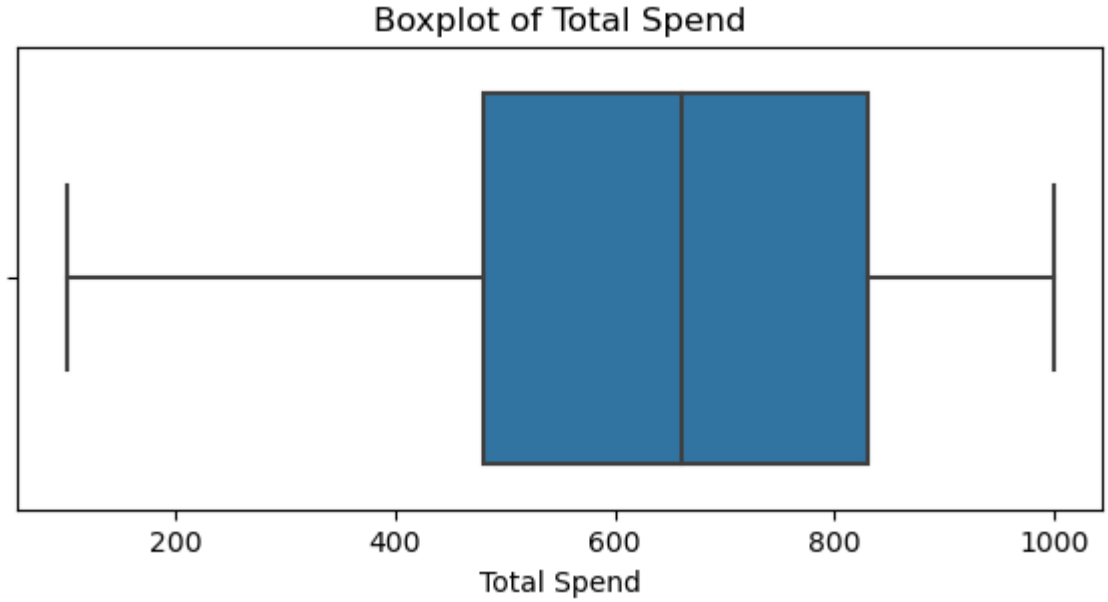

Histogram of Support Calls
Frequency
70000
60000
50000
40000
30000
20000
10000
0
0
2
4
6
8
10
Support Calls

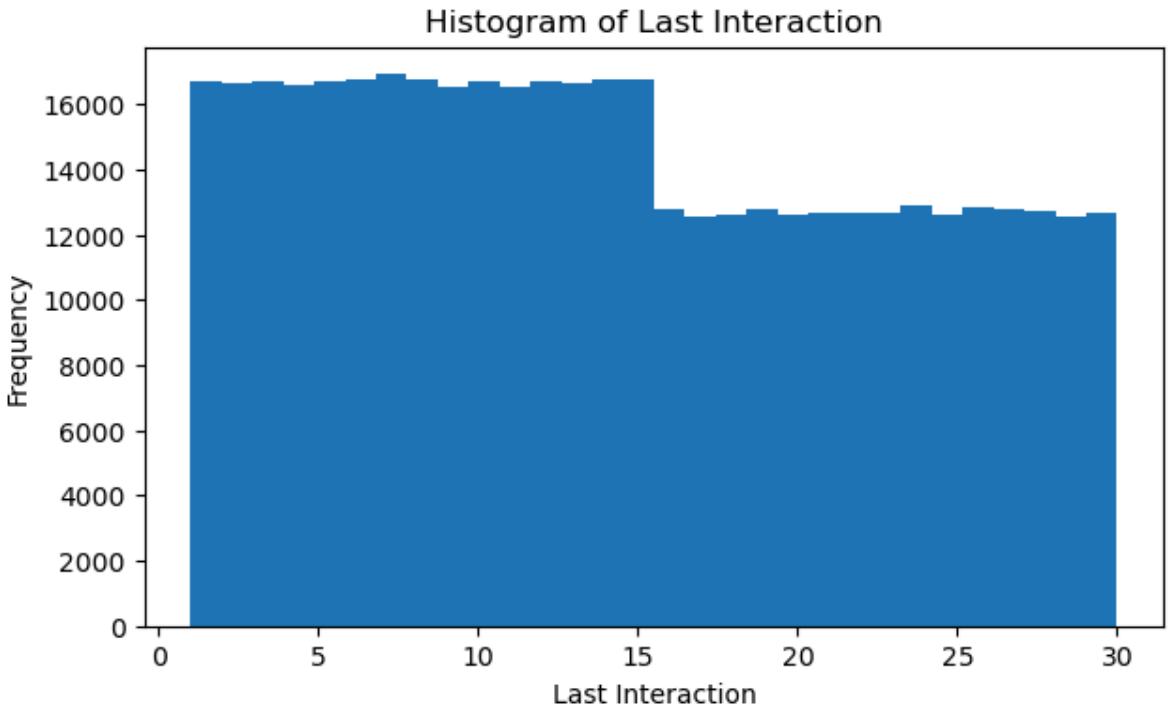

Boxplot of Support Calls
0
2
4
6
8
10
Support Calls

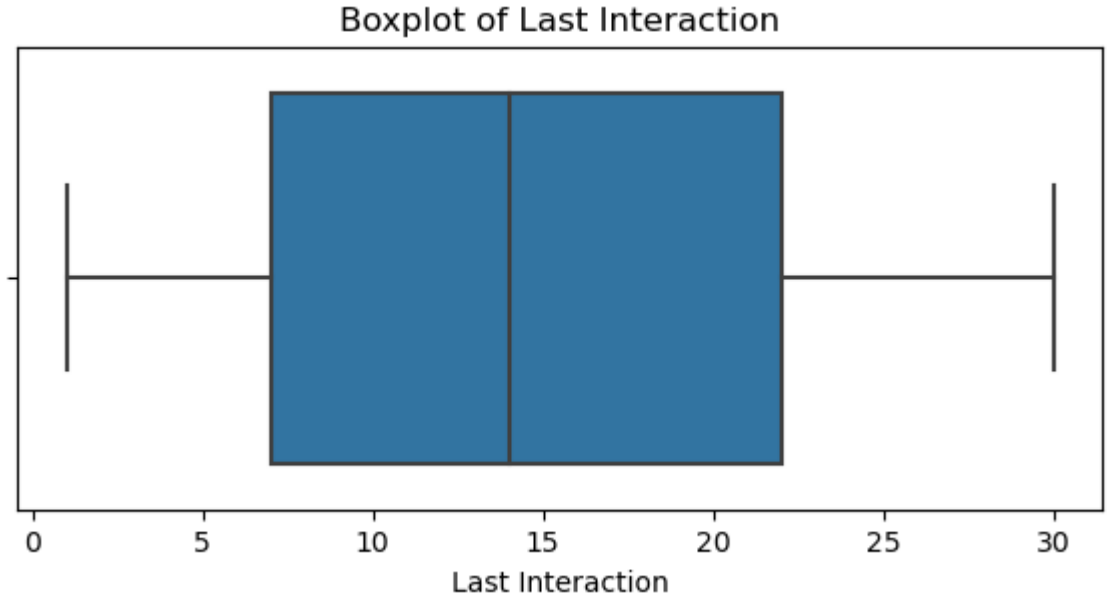

Histogram of Payment Delay
Frequency
17500
15000
12500
10000
7500
5000
2500
0
0
5
10
15
20
25
30
Payment Delay


Boxplot of Payment Delay
0
5
10
15
20
25
30
Payment Delay


Histogram of Total Spend
Frequency
20000
17500
15000
12500
10000
7500
5000
2500
0
200
400
600
800
1000
Total Spend


Boxplot of Total Spend
200
400
600
800
1000
Total Spend


Histogram of Last Interaction
Frequency
16000
14000
12000
10000
8000
6000
4000
2000
0
0
5
10
15
20
25
30
Last Interaction


Boxplot of Last Interaction
0
5
10
15
20
25
30
Last Interaction

```
============================================================
OUTLIER DETECTION USING IQR
============================================================

Feature: Age
Q1: 29.0
Q3: 48.0
IQR: 19.0
Lower Bound: 0.5
Upper Bound: 76.5
Number of Outliers: 0

Feature: Tenure
Q1: 16.0
Q3: 46.0
IQR: 30.0
Lower Bound: -29.0
Upper Bound: 91.0
Number of Outliers: 0

Feature: Usage Frequency
Q1: 9.0
Q3: 23.0
IQR: 14.0
Lower Bound: -12.0
Upper Bound: 44.0
Number of Outliers: 0

Feature: Support Calls
Q1: 1.0
Q3: 6.0
IQR: 5.0
Lower Bound: -6.5
Upper Bound: 13.5
Number of Outliers: 0

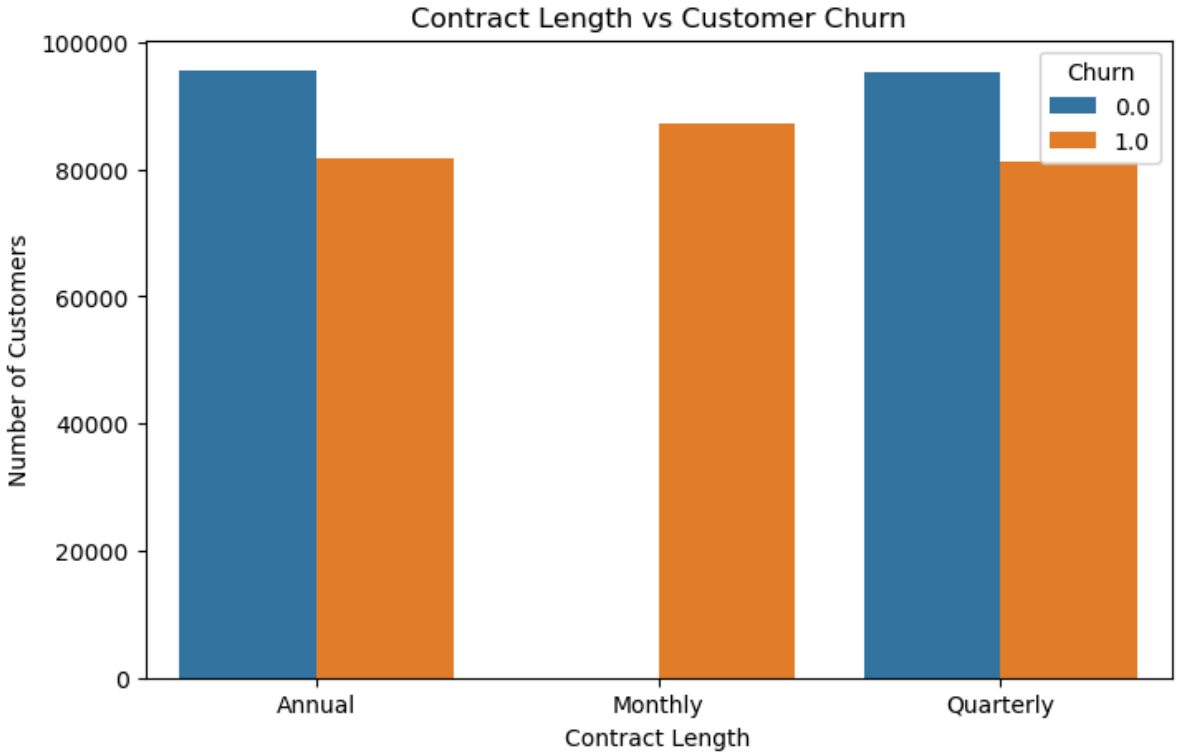
Feature: Payment Delay
Q1: 6.0
Q3: 19.0
IQR: 13.0
Lower Bound: -13.5
Upper Bound: 38.5
Number of Outliers: 0

Feature: Total Spend
Q1: 480.0
Q3: 830.0
IQR: 350.0
Lower Bound: -45.0
Upper Bound: 1355.0
Number of Outliers: 0

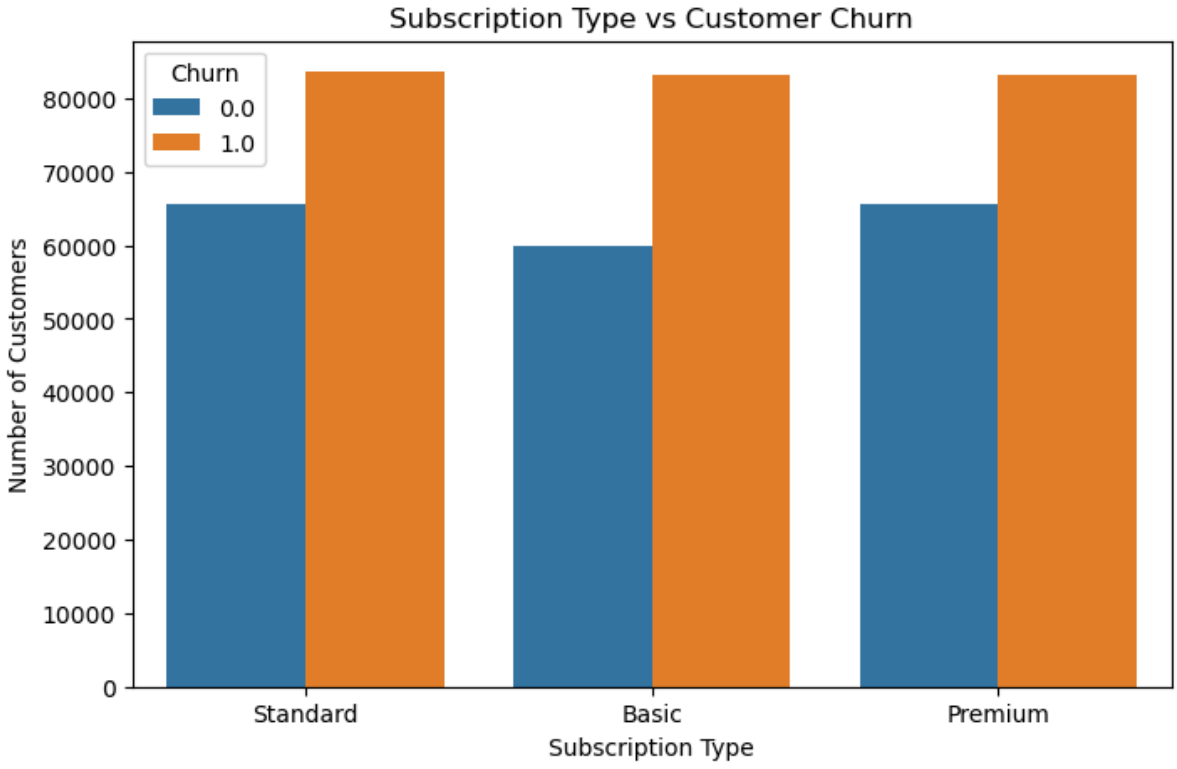
Feature: Last Interaction
Q1: 7.0
Q3: 22.0
IQR: 15.0
Lower Bound: -15.5
Upper Bound: 44.5
Number of Outliers: 0

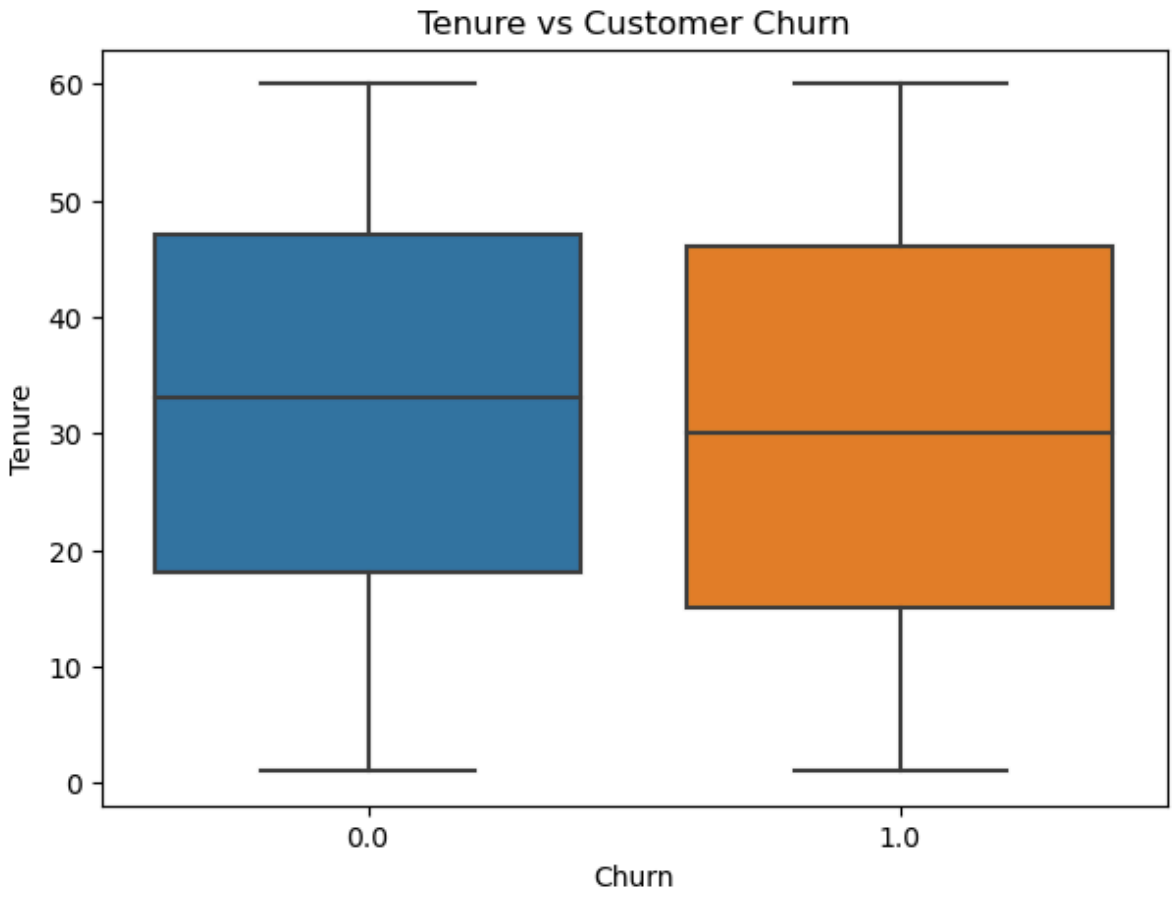
============================================================
CONTRACT LENGTH VS CHURN
============================================================
Churn                   0.0         1.0
```

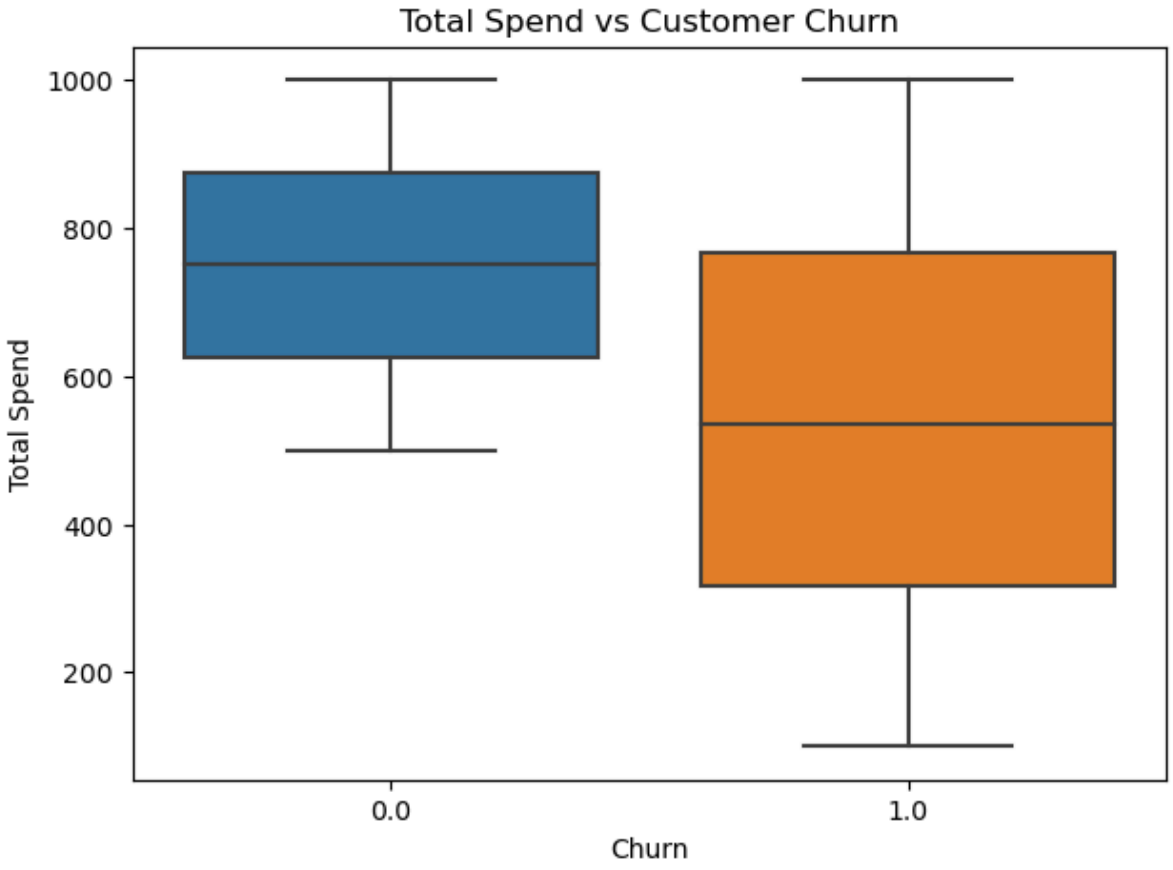
```
Contract Length
Annual            53.923859   46.076141
Monthly            0.000000  100.000000
Quarterly         53.974395   46.025605
```

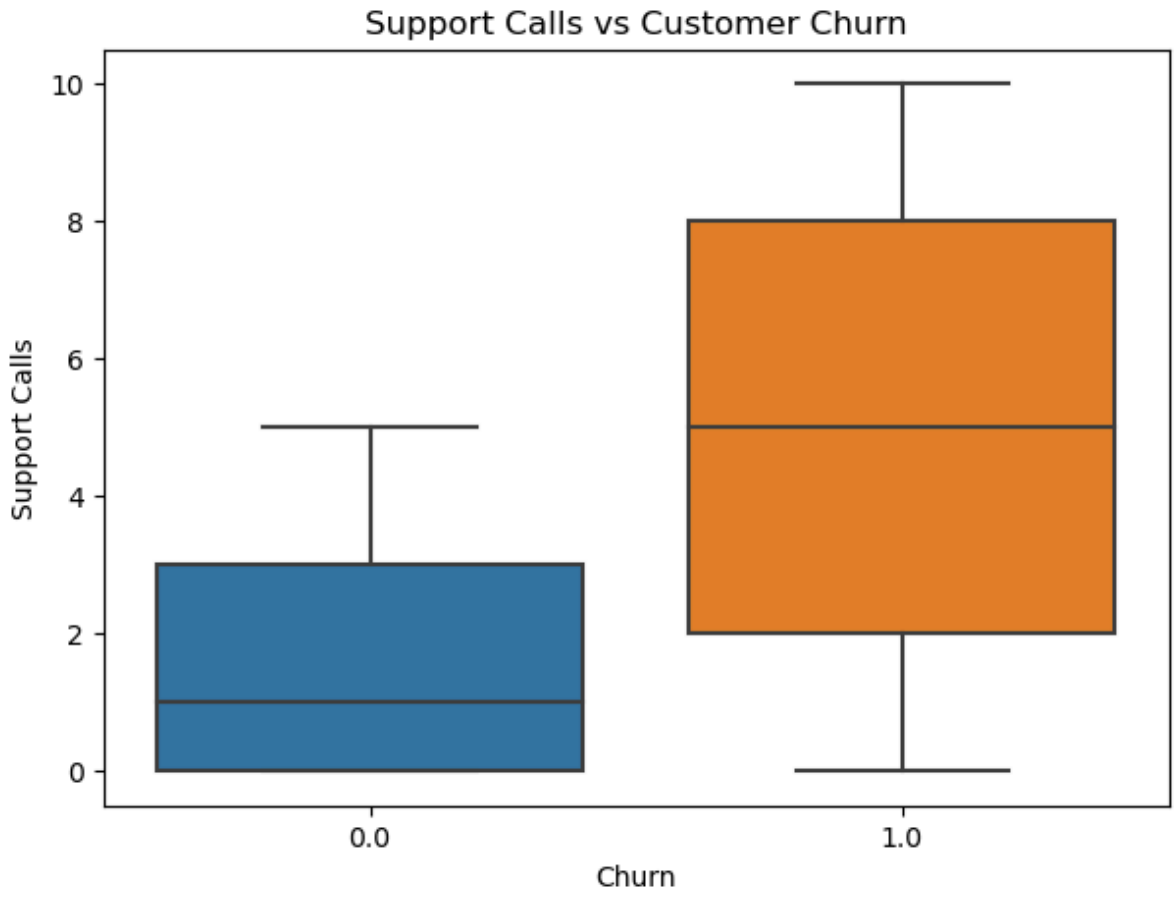


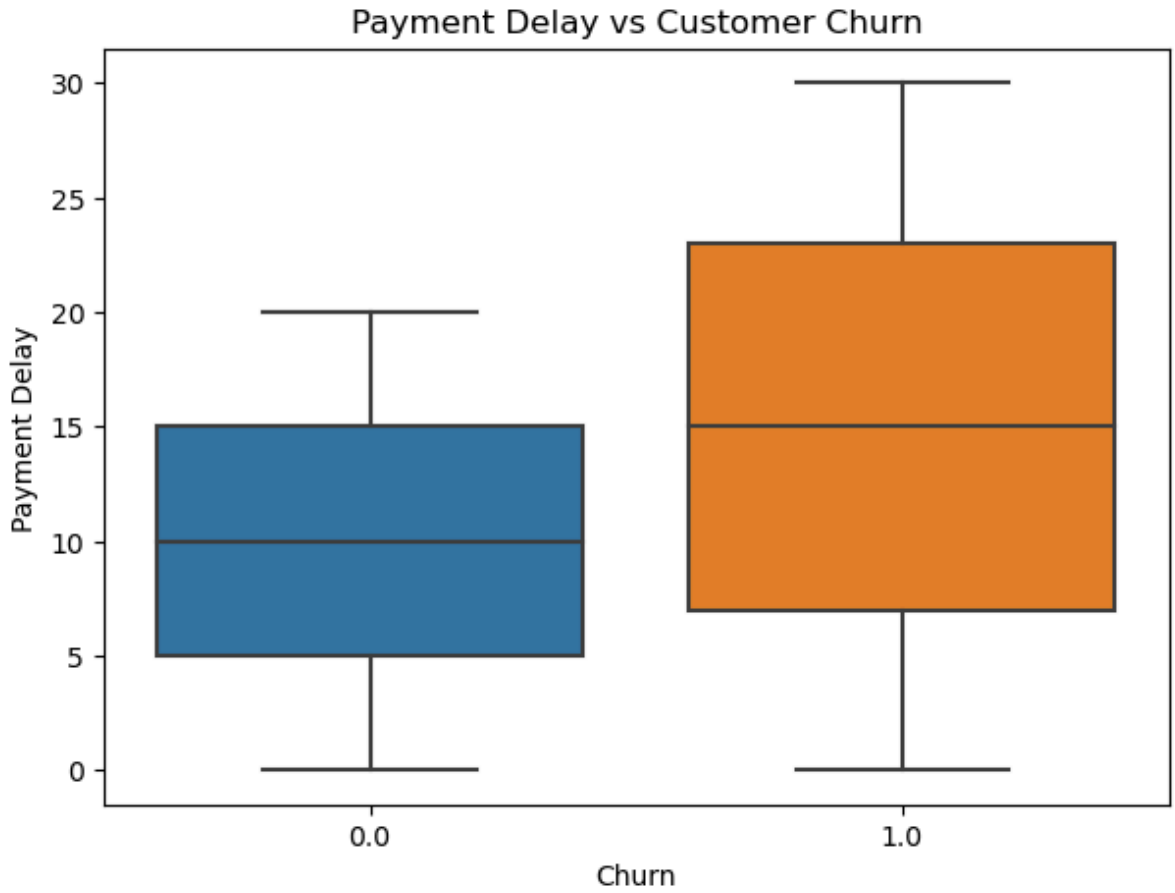

```
============================================================
SUBSCRIPTION TYPE VS CHURN
============================================================
Churn                     0.0        1.0
Subscription Type
Basic               41.821767  58.178233
Premium             44.058300  55.941700
Standard            43.930047  56.069953
```



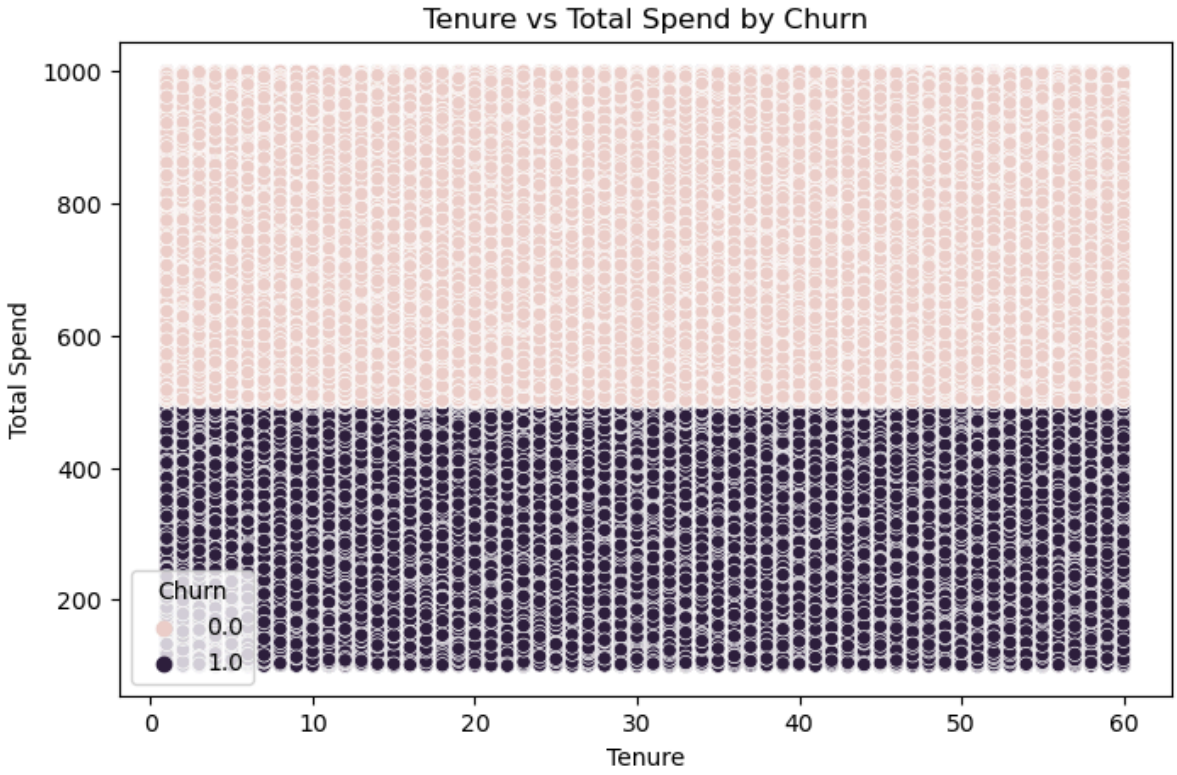

Tenure vs Customer Churn
Tenure
60
50
40
30
20
10
0
0.0
1.0
Churn


Total Spend vs Customer Churn
Total Spend
1000
800
600
400
200
0.0
1.0
Churn






```
C:\Users\DELL\anaconda3\lib\site-packages\IPython\core\pylabtools.py:152: UserWarn
ing: Creating legend with loc="best" can be slow with large amounts of data.
  fig.canvas.print_figure(bytes_io, **kw)
```


Tenure vs Total Spend by Churn
Total Spend
1000
800
600
400
200
Churn
0.0
1.0
0
10
20
30
40
50
60
Tenure

```
============================================================
ANALYTICAL QUESTIONS
============================================================

1. CONTRACT TYPE WITH HIGHEST CHURN
Churn                   0.0         1.0
Contract Length
Annual           53.923859   46.076141
Monthly           0.000000  100.000000
Quarterly        53.974395   46.025605
Contract Length with highest churn: Monthly
Churn Rate: 100.0 %

2. SUBSCRIPTION TYPE WITH HIGHEST CHURN
Churn                     0.0        1.0
Subscription Type
Basic               41.821767  58.178233
Premium             44.058300  55.941700
Standard            43.930047  56.069953
Subscription Type with highest churn: Basic

3. TENURE AND CHURN
Churn
0.0    32.281754
1.0    30.473598
Name: Tenure, dtype: float64

Median Tenure:
Churn
0.0    33.0
1.0    30.0
Name: Tenure, dtype: float64

4. AVERAGE TOTAL SPEND
Subscription Type
Standard    633.132253
Premium     632.926463
Basic       628.673496
Name: Total Spend, dtype: float64

Average Total Spend by Contract Length:
Contract Length
Annual       651.696325
Quarterly    651.426516
Monthly      550.618048
Name: Total Spend, dtype: float64

5. CHURN BY SUBSCRIPTION TYPE
Churn                     0.0        1.0
Subscription Type
Basic               41.821767  58.178233
Premium             44.058300  55.941700
Standard            43.930047  56.069953

6. TOTAL SPEND AND CHURN
Churn
0.0    749.953111
1.0    541.285528
Name: Total Spend, dtype: float64

Median Total Spend by Churn:
Churn
0.0    749.98
1.0    534.00
```

```
Name: Total Spend, dtype: float64

7. POTENTIAL STRONG INDICATORS OF CHURN

Average Support Calls by Churn:
Churn
0.0    1.586418
1.0    5.144861
Name: Support Calls, dtype: float64

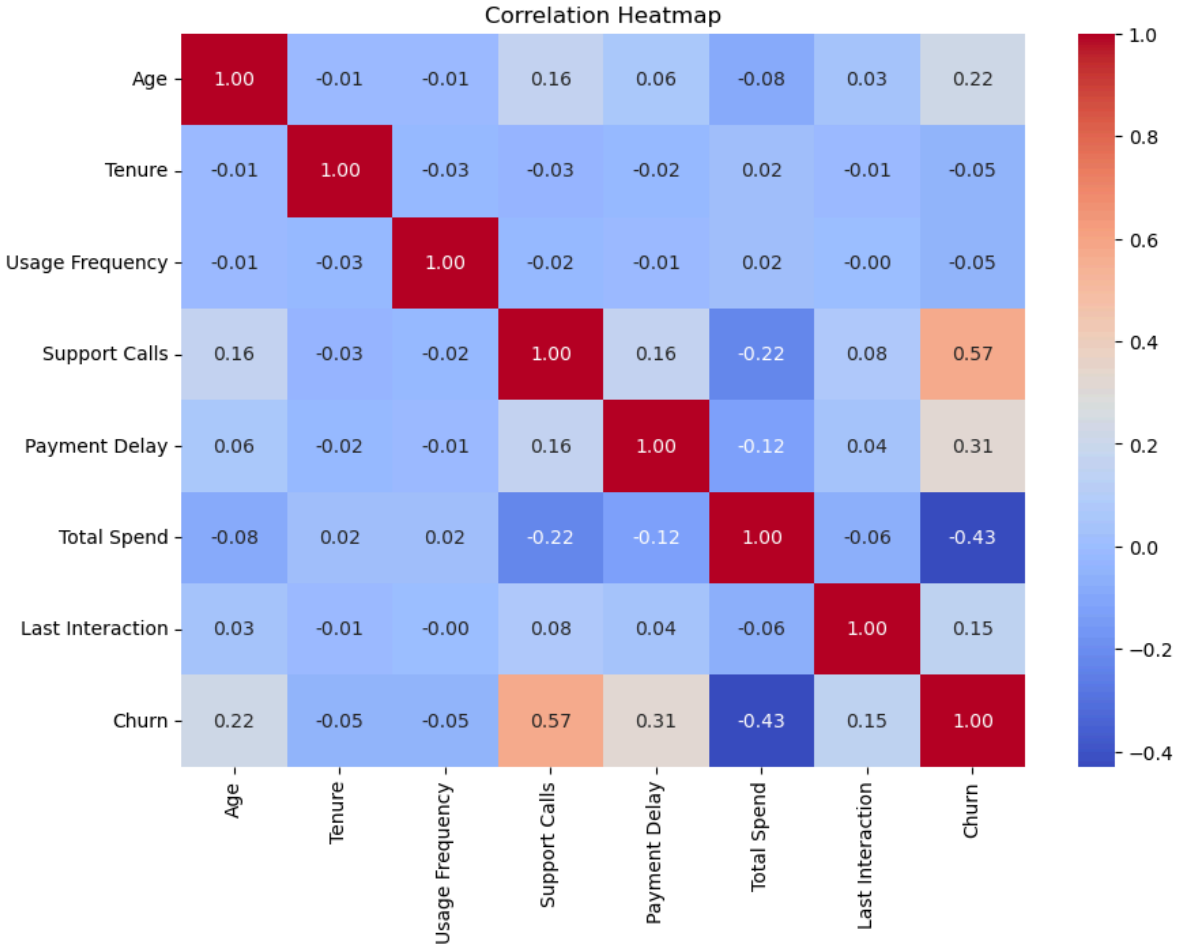
Average Payment Delay by Churn:
Churn
0.0    10.015500
1.0    15.217729
Name: Payment Delay, dtype: float64

Average Tenure by Churn:
Churn
0.0    32.281754
1.0    30.473598
Name: Tenure, dtype: float64

Average Usage Frequency by Churn:
Churn
0.0    16.260552
1.0    15.461658
Name: Usage Frequency, dtype: float64

Average Total Spend by Churn:
Churn
0.0    749.953111
1.0    541.285528
Name: Total Spend, dtype: float64

============================================================
CORRELATION WITH CHURN
============================================================
Churn               1.000000
Support Calls       0.574267
Payment Delay       0.312129
Age                 0.218394
Last Interaction    0.149616
Usage Frequency    -0.046101
Tenure             -0.051919
Total Spend        -0.429355
Name: Churn, dtype: float64
```


Correlation Heatmap
Age
Tenure
Usage Frequency
Support Calls
Payment Delay
Total Spend
Last Interaction
Churn
1.00 -0.01 -0.01 0.16 0.06 -0.08 0.03 0.22
-0.01 1.00 -0.03 -0.03 -0.02 0.02 -0.01 -0.05
-0.01 -0.03 1.00 -0.02 -0.01 0.02 -0.00 -0.05
0.16 -0.03 -0.02 1.00 0.16 -0.22 0.08 0.57
0.06 -0.02 -0.01 0.16 1.00 -0.12 0.04 0.31
-0.08 0.02 0.02 -0.22 -0.12 1.00 -0.06 -0.43
0.03 -0.01 -0.00 0.08 0.04 -0.06 1.00 0.15
0.22 -0.05 -0.05 0.57 0.31 -0.43 0.15 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

```
============================================================
TRAIN TEST SPLIT
============================================================

Missing values in Churn:
1

Churn values:
1.0    249999
0.0    190833
Name: Churn, dtype: int64

Missing values in target after cleaning:
0

Training Features Shape:
(352665, 12)

Testing Features Shape:
(88167, 12)

Training Target Shape:
(352665,)

Testing Target Shape:
(88167,)

============================================================
EXPERIMENT COMPLETED SUCCESSFULLY
============================================================

Final Processed Dataset Shape:
(440832, 13)

Training Dataset Shape:
(352665, 12)

Testing Dataset Shape:
(88167, 12)

File Saved Successfully At:
C:\Users\DELL\Downloads\CustomerChurn_Preprocessed.csv
```

In [ ]: